

```
import random

# Part A - Machine Initialization
machines = ['A', 'B', 'C']

N_play = 10
epsilon = 0.2

print("Machines:", machines)
print("Number of plays:", N_play)
print("Epsilon:", epsilon)
```

```
Machines: ['A', 'B', 'C']
Number of plays: 10
Epsilon: 0.2
```

```
# Part B - Initialize N, R and Q

N = {'A': 0, 'B': 0, 'C': 0}      # Number of times played
R = {'A': 0, 'B': 0, 'C': 0}      # Total reward
Q = {'A': 0.0, 'B': 0.0, 'C': 0.0} # Estimated reward

print("Initial N:", N)
print("Initial R:", R)
print("Initial Q:", Q)
```

```
Initial N: {'A': 0, 'B': 0, 'C': 0}
Initial R: {'A': 0, 'B': 0, 'C': 0}
Initial Q: {'A': 0.0, 'B': 0.0, 'C': 0.0}
```

```
# Suggested Experiment - Reward Patterns
```

```
rewards = {
    'A': [3, 4, 2, 5],
    'B': [6, 7, 5, 8],
    'C': [4, 3, 5, 4]
}

# To keep track of the next reward for each machine
reward_index = {
    'A': 0,
    'B': 0,
    'C': 0
}

print("Reward Patterns:")
print("A:", rewards['A'])
print("B:", rewards['B'])
print("C:", rewards['C'])
```

```
Reward Patterns:
A: [3, 4, 2, 5]
B: [6, 7, 5, 8]
C: [4, 3, 5, 4]
```

```
# Part C, D and E
# Modified Epsilon-Greedy Strategy

# Initialize variables for the simulation
# This ensures a fresh start if the cell is run multiple times
N = {'A': 0, 'B': 0, 'C': 0}
R = {'A': 0, 'B': 0, 'C': 0}
Q = {'A': 0.0, 'B': 0.0, 'C': 0.0}

reward_index = {
    'A': 0,
    'B': 0,
    'C': 0
}

total_reward = 0

for play in range(1, N_play + 1):
```

```

print("\n=====")
print("PLAY", play)
print("=====")

# Step 1: Generate random number
r = random.random()
print("Random number =", round(r, 3))

# Step 2: Find machine with highest Q-value
highest_Q = max(Q.values())

best_machine = max(Q, key=Q.get)

# Step 3: Modified Epsilon-Greedy Decision
if r < epsilon:
    # Exploration
    # Select a machine other than best machine

    other_machines = [
        m for m in machines
        if m != best_machine
    ]

    selected_machine = random.choice(other_machines)

    print("Decision = Exploration")

else:
    # Exploitation
    # Select machine with highest Q-value

    selected_machine = best_machine

    print("Decision = Exploitation")

print("Selected Machine =", selected_machine)

# Step 4: Get reward from given reward pattern
index = reward_index[selected_machine]

reward = rewards[selected_machine][index]

# Update reward_index to loop back to the beginning of the pattern
reward_index[selected_machine] = (reward_index[selected_machine] + 1) % len(rewards[selected_machine])

print("Reward =", reward)

# Step 5: Update N
N[selected_machine] += 1

# Step 6: Update R
R[selected_machine] += reward

# Step 7: Update Q using incremental sample average
old_Q = Q[selected_machine]

Q[selected_machine] = (
    old_Q +
    (reward - old_Q) / N[selected_machine]
)

# Step 8: Update total reward
total_reward += reward

# Step 9: Display Q calculation

print("\nQ-value Calculation:")

print(
    f"Q({selected_machine}) = "
    f"{old_Q:.2f} + "

```

```

        f"({reward} - {old_Q:.2f}) / "
        f"{N[selected_machine]}"
    )

    print(
        f"Q({selected_machine}) = "
        f"{Q[selected_machine]:.2f}"
    )

```

```

=====
PLAY 1
=====
Random number = 0.946
Decision = Exploitation
Selected Machine = A
Reward = 3

Q-value Calculation:
 $Q(A) = 0.00 + (3 - 0.00) / 1$ 
 $Q(A) = 3.00$ 

=====
PLAY 2
=====
Random number = 0.311
Decision = Exploitation
Selected Machine = A
Reward = 4

Q-value Calculation:
 $Q(A) = 3.00 + (4 - 3.00) / 2$ 
 $Q(A) = 3.50$ 

=====
PLAY 3
=====
Random number = 0.241
Decision = Exploitation
Selected Machine = A
Reward = 2

Q-value Calculation:
 $Q(A) = 3.50 + (2 - 3.50) / 3$ 
 $Q(A) = 3.00$ 

=====
PLAY 4
=====
Random number = 0.051
Decision = Exploration
Selected Machine = B
Reward = 6

Q-value Calculation:
 $Q(B) = 0.00 + (6 - 0.00) / 1$ 
 $Q(B) = 6.00$ 

=====
PLAY 5
=====
Random number = 0.758
Decision = Exploitation
Selected Machine = B
Reward = 7

Q-value Calculation:

```

